

ZERODAY REAPERS

GOVERNANCE, RISK & COMPLIANCE INTERNSHIP | WEEK 04

THREATS TO BACKLOG: STRIDE THREAT MODELING, LIVE VALIDATION, AND STRUCTURED REMEDIATION PLANNING

Field	Detail
Task ID	ZDR-W4-GRC-24GB
Assessment Type	Threats to Backlog: STRIDE Threat Modeling, Live Validation, and Structured Remediation Planning
Platform	Docker Container on BUN VM (Ubuntu 22.04)
Target Application	OWASP Juice Shop
Endpoint	http://127.0.0.1:3000 IP: 192.168.136.131
Prepared By	Abdullah Zubair
Report Date	14 September 2026

Table of Contents

1. Executive Summary	3
2. Introduction and Scope	4
3. Methodology	5
4. Environment and Infrastructure Setup	7
5. Data Flow Diagram (DFD)	10
6. STRIDE Threat Register	14
7. DREAD Risk Scoring	19
8. Live Threat Validations	22
9. Remediation Backlog	32
10. Sprint Plan and Roadmap	40
11. Risk Treatment Decisions	43
12. Evidence Register	44
13. Conclusion	46

1. Executive Summary

This report presents the findings, analysis, and remediation planning arising from a structured threat modeling exercise conducted against OWASP Juice Shop version 17.x, deployed as a Docker container on a single Ubuntu 22.04 virtual machine designated BUN. The exercise was performed as part of the Week 4 Governance, Risk, and Compliance (GRC) assignment and follows recognised industry-standard methodologies including STRIDE threat classification, DREAD risk scoring, OWASP Top 10 alignment, MITRE ATT&CK for Enterprise mapping, and structured sprint-based remediation planning.

OWASP Juice Shop is an intentionally vulnerable web application designed to represent a realistic modern web stack comprising an Angular Single Page Application (SPA) frontend, a Node.js/Express REST API backend, and an SQLite database. Its deliberate inclusion of OWASP Top 10 vulnerabilities makes it an appropriate and controlled target for this threat modeling and validation exercise.

Key findings and outcomes of this exercise are summarised below:

Metric	Value
Total threats identified (STRIDE)	12
Threats classified Critical (DREAD ≥ 8.0)	4
Threats classified High (DREAD 6.0-7.9)	7
Threats classified Medium (DREAD 4.0-5.9)	1
Threats validated with live evidence	6 of 6 targeted (100%)
Remediation backlog items created	12 (REM-01 to REM-12)
Sprint 1 items (Critical, Weeks 1-4)	4
Sprint 2 items (High Risk, Weeks 5-8)	4
Sprint 3 items (Defence in Depth, Weeks 9-12)	4
Total estimated remediation effort	35 developer-days
OWASP Top 10 categories covered	6
MITRE ATT&CK tactics mapped	5
CWE identifiers assigned	12

All six targeted threats were confirmed through live exploitation against the running application, producing photographic evidence (EV-14 through EV-19). The most severe finding was a SQL injection vulnerability (THR-01, DREAD 9.0) in the product search endpoint that allows a remote unauthenticated attacker to read the entire database. An equally severe broken access control finding (THR-11, DREAD 9.0) allows any registered user to access administrative functionality without privilege escalation. Both represent material risks that would result in a reportable data breach under most data protection frameworks including GDPR and PDPL.

The remediation backlog is structured to address all Critical findings within the first sprint (Weeks 1-4), ensuring the most material risks are mitigated before High and Medium items are addressed in subsequent sprints. Each backlog item includes acceptance criteria to provide unambiguous definition-of-done for the development and security teams.

2. Introduction and Scope

2.1 Purpose

The purpose of this report is to document the end-to-end threat modeling lifecycle applied to a representative modern web application. Specifically, this report aims to:

- Construct a Data Flow Diagram (DFD) that accurately reflects the application architecture and data movements.
- Apply STRIDE threat classification to enumerate plausible threats across all DFD elements and trust boundaries.
- Score all identified threats using the DREAD risk scoring model to enable prioritised treatment.
- Validate a subset of high-priority threats through controlled live exploitation, producing evidence-backed findings.
- Produce a structured remediation backlog with effort estimates, acceptance criteria, and sprint assignments.
- Demonstrate GRC competency through structured risk identification, analysis, evaluation, and treatment planning.

2.2 Scope

The scope of this exercise is limited to the OWASP Juice Shop application as deployed on the BUN virtual machine. Only one virtual machine was used throughout the entire assignment. The application was accessed exclusively at <http://127.0.0.1:3000> from within the same VM. No external network targets, live production systems, or third-party services were accessed or tested. The following components fall within scope:

- Juice Shop Angular SPA frontend (client-side JavaScript, routing, DOM handling)
- Juice Shop REST API (Node.js/Express, all documented and undocumented endpoints)
- SQLite database backend (data storage, query handling)
- Authentication and session management subsystem (JWT-based)
- File upload functionality
- Admin panel and role-based access control

The following are explicitly out of scope:

- Physical infrastructure and hypervisor layer
- Network perimeter devices and firewalls
- Third-party CDN or cloud services
- Denial-of-service testing beyond controlled rate-limit validation
- Social engineering or phishing simulations

2.3 Intended Audience

This report is intended for academic assessment. However, it is structured to be readable by a mixed audience including security engineers responsible for implementing remediations, development team leads who will manage sprint work, risk and compliance officers who require a risk register and treatment plan, and management stakeholders who require an executive summary of material risk exposure.

3. Methodology

3.1 Threat Modeling Framework

This exercise follows the Microsoft STRIDE threat modeling framework, which was chosen for its wide industry adoption, its suitability for web application threat modeling, and its direct integration with OWASP Threat Dragon — the open-source tooling used in this assessment. STRIDE is an acronym representing six threat categories:

Category	Description	Violated Property
Spoofing	Impersonating another user, system, or entity	Authentication
Tampering	Unauthorised modification of data or code	Integrity
Repudiation	Denying actions without the ability to prove otherwise	Non-repudiation
Information Disclosure	Exposing information to unauthorised parties	Confidentiality
Denial of Service	Degrading or destroying service availability	Availability
Elevation of Privilege	Gaining capabilities beyond what is authorised	Authorisation

3.2 Risk Scoring: DREAD Model

Each identified threat is scored using the DREAD model, which produces a composite risk score from five dimensions. Each dimension is scored on a scale of 1 (low) to 10 (high), and the final DREAD score is the arithmetic mean of the five dimension scores.

Dimension	Question Asked	Scale
Damage	How bad would an attack be if it succeeded?	1 = Minimal, 10 = Catastrophic
Reproducibility	How easy is it to reproduce the attack?	1 = Very difficult, 10 = Trivially reproducible
Exploitability	How much work is needed to launch the attack?	1 = Requires expert skills, 10 = No skill required
Affected Users	How many users are impacted?	1 = Single user, 10 = All users
Discoverability	How easy is it to find the vulnerability?	1 = Very hard, 10 = Visible in browser

The risk thresholds applied in this report are as follows:

Risk Level	DREAD Score Range	Required Sprint
Critical	8.0 - 10.0	Sprint 1 (Weeks 1-4): Immediate remediation
High	6.0 - 7.9	Sprint 2 (Weeks 5-8): Remediate within 8 weeks
Medium	4.0 - 5.9	Sprint 3 (Weeks 9-12): Remediate within 12 weeks
Low	1.0 - 3.9	Backlog: Address in next planning cycle

3.3 Validation Approach

Live validation was conducted for six threats selected on the basis of exploitability and risk level. Each validation followed a structured approach: (1) define the test objective and expected outcome; (2) execute the test using standard tools available on the BUN VM (curl, browser, Burp Suite-equivalent manual testing); (3) capture a screenshot as primary evidence; (4) document the actual outcome against the expected outcome; and (5) confirm or deny the threat. All validations returned a CONFIRMED status, meaning the vulnerability was demonstrated in practice.

It is important to note that these tests were conducted in a controlled, isolated environment against an intentionally vulnerable application. No real user data was at risk. The tests were non-destructive and reversible.

3.4 Remediation Planning Approach

Remediation backlog items were created following a structured template aligned with Agile security backlog practices. Each item specifies: the linked threat identifier; the DREAD score and risk priority; the affected component; a plain-language description of the vulnerability; the recommended remediation action with implementation guidance; measurable acceptance criteria; effort estimate in developer-days; sprint assignment; dependencies; OWASP Top 10 category; CWE identifier; MITRE ATT&CK technique; and evidence reference.

Sprint assignments are based on risk priority (DREAD score) and inter-item dependencies. Items in Sprint 1 must be completed before Sprint 2 items that depend on them (e.g., REM-09 Content Security Policy in Sprint 3 depends on REM-05 DOM XSS encoding fixes in Sprint 2 being in place first).

4. Environment and Infrastructure Setup

4.1 Virtual Machine Specification (BUN)

A single virtual machine named BUN was used for the entirety of this assignment. This deliberate single-VM strategy was chosen to simplify the network topology, reduce complexity, and ensure all testing remained within a controlled and isolated environment. The VM specification is detailed below.

Component	Detail
VM Name	BUN
Operating System	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
Kernel Version	6.8.0-136-generic
IP Address	192.168.136.131/24
Network Interface	ens33
Timezone	Asia/Riyadh (UTC+3)
Docker Version	29.1.3
Container Name	juice-shop
Container ID	3d868f1376d0
Docker Image	bkimminich/juice-shop
Port Binding	127.0.0.1:3000 -> 3000/tcp (loopback only)
Threat Dragon Version	v2.6.2-latest (web app, local-user session)
Application Access URL	http://127.0.0.1:3000

4.2 Deployment Process

OWASP Juice Shop was deployed using Docker. The image was pulled from Docker Hub using the official `bkimminich/juice-shop` image. The container was launched with a loopback-only port binding (`127.0.0.1:3000`) to prevent the application from being accessible on the VM's external network interface, reducing the attack surface to the local VM only. This is consistent with responsible disclosure practices for intentionally vulnerable applications.

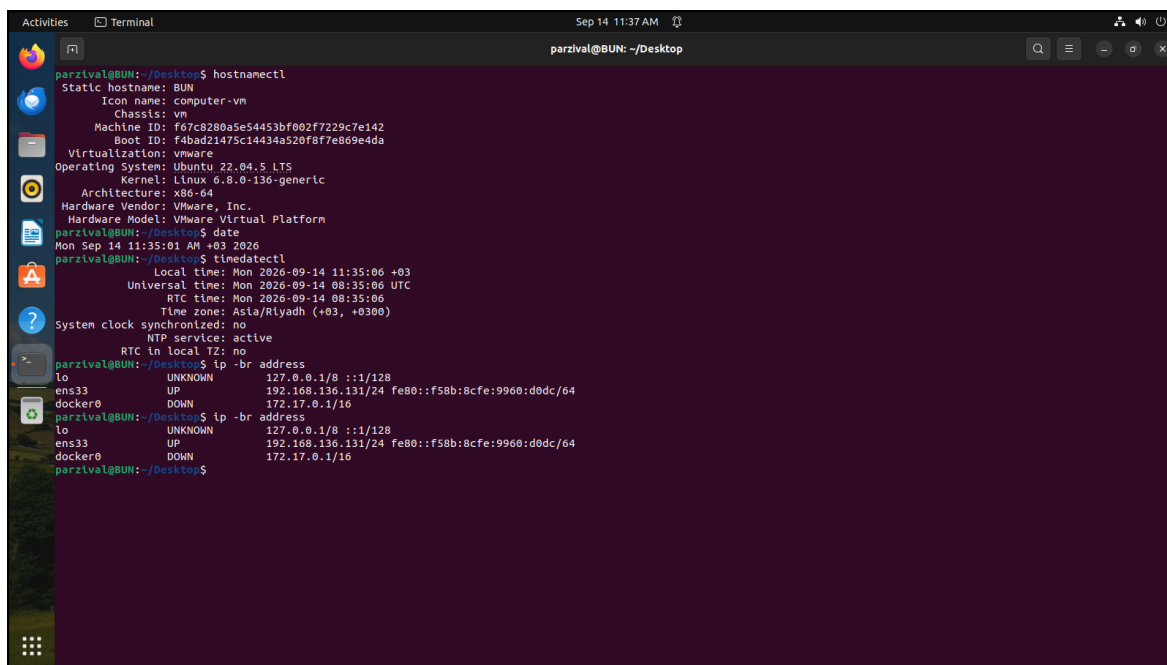
OWASP Threat Dragon was used as the threat modeling tooling. It was accessed via its hosted web interface at www.threatdragon.com using a local-user session (no account required). The threat model data was stored in the browser's local storage and exported as a JSON file for archiving.

4.3 Baseline Evidence

The following screenshots document the baseline state of the BUN VM and the Juice Shop deployment. These images serve as the foundational evidence set (EV-01 through EV-11) establishing that the environment was correctly configured prior to threat modeling and validation activities.

EV-01 — BUN VM System Baseline:

The following screenshot captures the output of `uname -a`, `ip addr show`, and `date` commands on the BUN VM, confirming the OS version, kernel, IP address, and system clock state at the time of the exercise.



```
parzival@BUN: ~/Desktop$ hostnamectl
Static hostname: BUN
Icon name: computer-vm
Chassis: vm
Machine ID: f67c0280a5e54453bf002f7229c7e142
Boot ID: f4bad21475c14434a520f87e069e4da
Virtualization: vmware
Operating System: Ubuntu 22.04.5 LTS
Kernel: Linux 6.8.0-136-generic
Architecture: x86_64
Hardware Vendor: VMware, Inc.
Hardware Model: VMware Virtual Platform

parzival@BUN: ~/Desktop$ date
Mon Sep 14 11:35:01 AM +03 2026

parzival@BUN: ~/Desktop$ timedatectl
Local time: Mon 2026-09-14 11:35:00 +03
Universal time: Mon 2026-09-14 08:35:00 UTC
RTC time: Mon 2026-09-14 08:35:00
Time zone: Asia/Riyadh (+03, +0300)
System clock synchronized: no
NTP service: active
RTC in local TZ: no

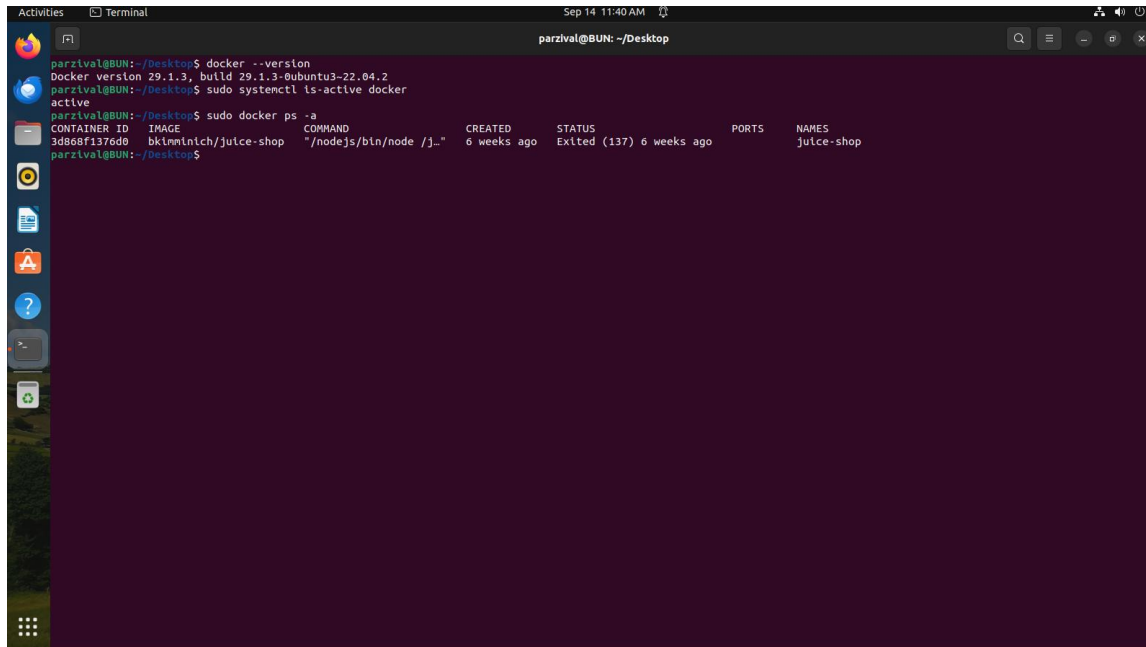
parzival@BUN: ~/Desktop$ ip -br address
lo UNKNOWN 127.0.0.1/8 ::1/128
ens33 UP 192.168.136.131/24 fe80::f58b:8cfe:9960:d0dc/64
docker0 DOWN 172.17.0.1/16

parzival@BUN: ~/Desktop$ ip -br address
lo UNKNOWN 127.0.0.1/8 ::1/128
ens33 UP 192.168.136.131/24 fe80::f58b:8cfe:9960:d0dc/64
docker0 DOWN 172.17.0.1/16
```

EV-01: BUN VM system baseline — `uname`, `ip addr`, `date` output

EV-02 — Docker Baseline:

Docker version and running container confirmed via `docker version` and `docker ps` commands.

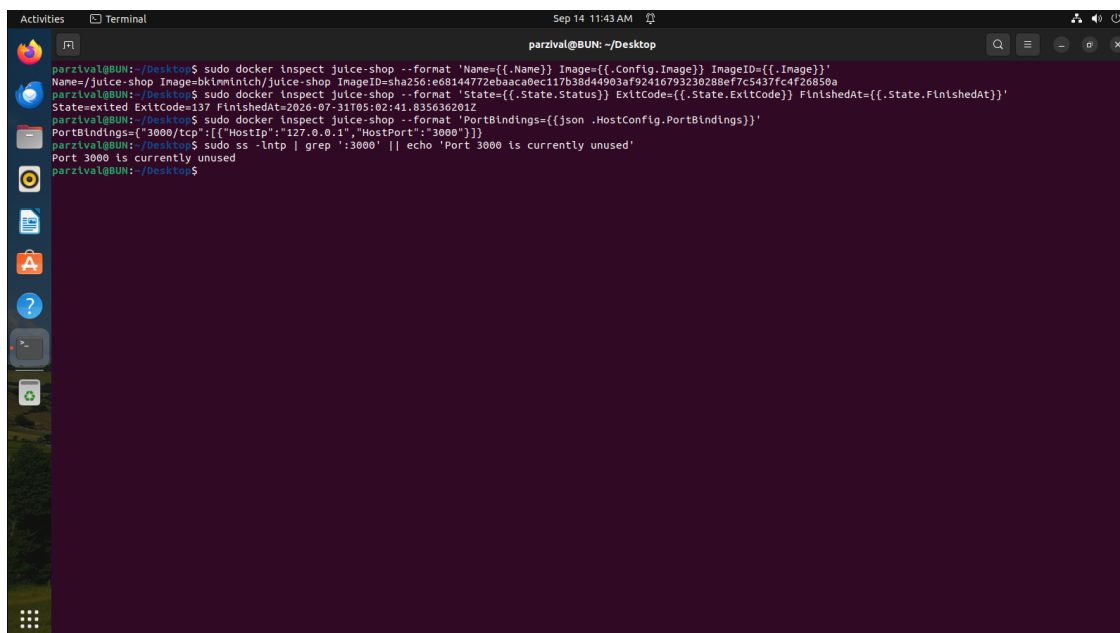
A terminal window titled 'parzival@BUN: ~/Desktop' showing the output of 'docker --version' and 'sudo systemctl is-active docker'. The first command shows Docker version 29.1.3, build 29.1.3-0ubuntu3-22.04.2. The second command shows 'active'. Below this, the output of 'sudo docker ps -a' is shown as a table with columns: CONTAINER ID, IMAGE, COMMAND, CREATED, STATUS, PORTS, and NAMES. The table contains one entry for container '3d868f1376de' running 'bkinminich/juice-shop' with command '"/nodejs/bin/node /j..."', created '6 weeks ago', status 'Exited (137) 6 weeks ago', and name 'juice-shop'.

```
parzival@BUN:~/Desktop$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3-22.04.2
parzival@BUN:~/Desktop$ sudo systemctl is-active docker
active
parzival@BUN:~/Desktop$ sudo docker ps -a
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS   NAMES
3d868f1376de   bkinminich/juice-shop  "/nodejs/bin/node /j..."  6 weeks ago   Exited (137) 6 weeks ago           juice-shop
parzival@BUN:~/Desktop$
```

EV-02: Docker baseline — version and container state

EV-03 — Container Inspection:

Detailed container configuration including port binding, environment variables, and image metadata.

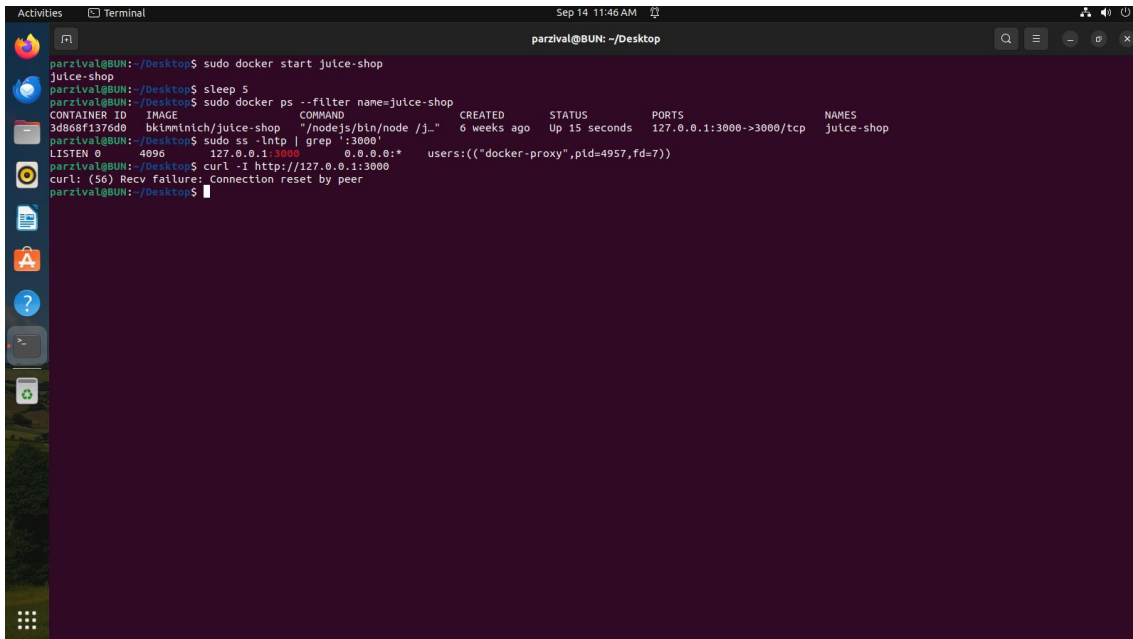
A terminal window titled 'parzival@BUN: ~/Desktop' showing the output of 'sudo docker inspect juice-shop --format' followed by three different format strings. The first format string shows Name, Image, and ImageID. The second format string shows State and ExitCode. The third format string shows PortBindings. The output for the third command shows 'PortBindings:{"3000/tcp":[{"HostIp":"127.0.0.1","HostPort":"3000"}]}' and a message 'Port 3000 is currently unused'.

```
parzival@BUN:~/Desktop$ sudo docker inspect juice-shop --format 'Name={{.Name}} Image={{.Config.Image}} ImageID={{.ImageID}}'
Name=/juice-shop Image=bkinminich/juice-shop ImageID=sha256:ed8144772ebaaca0ec117b38d44903af92416793230288ef7c5437fc4f26850a
parzival@BUN:~/Desktop$ sudo docker inspect juice-shop --format 'State={{.State.Status}} ExitCode={{.State.ExitCode}} FinishedAt={{.State.FinishedAt}}'
State=exited ExitCode=137 FinishedAt=2026-07-31T05:02:41.835630201Z
parzival@BUN:~/Desktop$ sudo docker inspect juice-shop --format 'PortBindings={{.HostConfig.PortBindings}}'
PortBindings={"3000/tcp":[{"HostIp":"127.0.0.1","HostPort":"3000"}]}
parzival@BUN:~/Desktop$ sudo ss -ltnp | grep ':3000' || echo 'Port 3000 is currently unused'
Port 3000 is currently unused
parzival@BUN:~/Desktop$
```

EV-03: docker inspect output for juice-shop container

EV-04 — Juice Shop Container Running State:

docker ps confirming the juice-shop container is up and listening on port 3000.

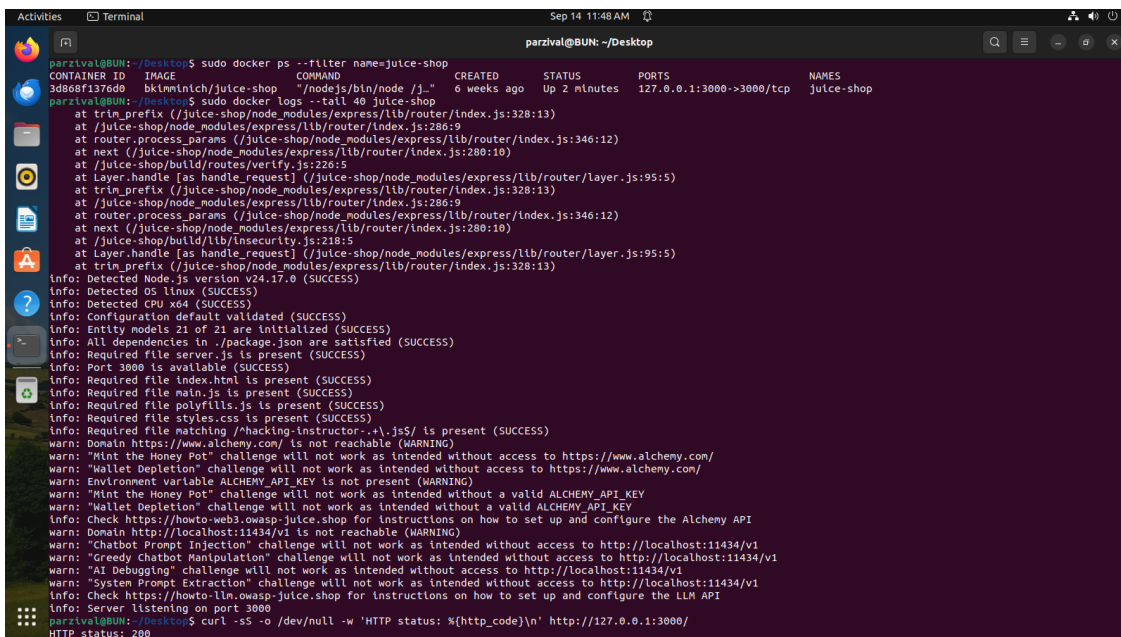


```
parzival@BUN: ~/Desktop
parzival@BUN:~/Desktop$ sudo docker start juice-shop
juice-shop
parzival@BUN:~/Desktop$ sleep 5
parzival@BUN:~/Desktop$ sudo docker ps --filter name=juice-shop
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                    NAMES
3d868f1376d9   bkmininch/juice-shop   "/nodejs/bin/node /j..." 6 weeks ago    Up 15 seconds  127.0.0.1:3000->3000/tcp  juice-shop
parzival@BUN:~/Desktop$ sudo ss -ltnp | grep ':3000'
LISTEN 0      4096      127.0.0.1:3000      0.0.0.0:*        users:((("docker-proxy",pid=4957,fd=7))
parzival@BUN:~/Desktop$ curl -I http://127.0.0.1:3000
curl: (56) Recv failure: Connection reset by peer
parzival@BUN:~/Desktop$
```

EV-04: docker ps showing juice-shop running

EV-05 — Application Readiness Check:

HTTP connectivity to the Juice Shop API confirmed, showing the application is responsive and ready.

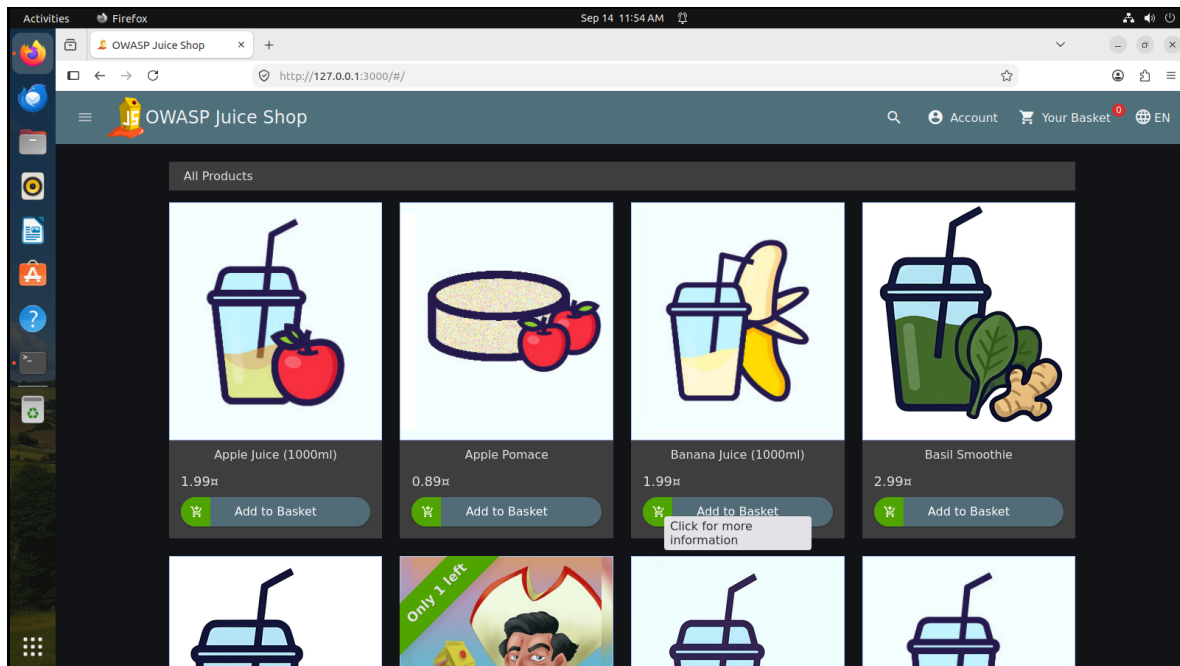


```
parzival@BUN:~/Desktop$ sudo docker ps --filter name=juice-shop
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                    NAMES
3d868f1376d9   bkmininch/juice-shop   "/nodejs/bin/node /j..." 6 weeks ago    Up 2 minutes  127.0.0.1:3000->3000/tcp  juice-shop
parzival@BUN:~/Desktop$ sudo docker logs --tail 40 juice-shop
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/routes/verify.js:226:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
at /juice-shop/node_modules/express/lib/router/index.js:286:9
at router.process_params (/juice-shop/node_modules/express/lib/router/index.js:346:12)
at next (/juice-shop/node_modules/express/lib/router/index.js:280:10)
at /juice-shop/build/lib/insecurity.js:210:5
at Layer.handle [as handle_request] (/juice-shop/node_modules/express/lib/router/layer.js:95:5)
at trim_prefix (/juice-shop/node_modules/express/lib/router/index.js:328:13)
Info: Detected Node.js version v24.17.0 (SUCCESS)
Info: Detected OS linux (SUCCESS)
Info: Detected CPU x64 (SUCCESS)
Info: Configuration default validated (SUCCESS)
Info: Entity models 21 of 21 are initialized (SUCCESS)
Info: All dependencies in ./package.json are satisfied (SUCCESS)
Info: Required file server.js is present (SUCCESS)
Info: Port 3000 is available (SUCCESS)
Info: Required file index.html is present (SUCCESS)
Info: Required file main.js is present (SUCCESS)
Info: Required file polyfills.js is present (SUCCESS)
Info: Required file styles.css is present (SUCCESS)
Info: Required file matching /hacking-instructor-*.js/ is present (SUCCESS)
warn: Domain https://www.alchemy.com/ is not reachable (WARNING)
warn: "Mint the Honey Pot" challenge will not work as intended without access to https://www.alchemy.com/
warn: "Wallet Depletion" challenge will not work as intended without access to https://www.alchemy.com/
warn: Environment variable ALCHEMY_API_KEY is not present (WARNING)
warn: "Mint the Honey Pot" challenge will not work as intended without a valid ALCHEMY_API_KEY
warn: "Wallet Depletion" challenge will not work as intended without a valid ALCHEMY_API_KEY
Info: Check https://howto-web3-owasp-juice.shop for instructions on how to set up and configure the Alchemy API
warn: Domain http://localhost:11434/v1 is not reachable (WARNING)
warn: "Chatbot Prompt Injection" challenge will not work as intended without access to http://localhost:11434/v1
warn: "Greedy Chatbot Manipulation" challenge will not work as intended without access to http://localhost:11434/v1
warn: "AI Debugging" challenge will not work as intended without access to http://localhost:11434/v1
warn: "System Prompt Extraction" challenge will not work as intended without access to http://localhost:11434/v1
Info: Check https://howto-llm-owasp-juice.shop for instructions on how to set up and configure the LLM API
Info: Server listening on port 3000
parzival@BUN:~/Desktop$ curl -s -o /dev/null -w 'HTTP status: %{http_code}\n' http://127.0.0.1:3000/
HTTP status: 200
```

EV-05: Juice Shop readiness — HTTP response confirmed

EV-06 — Juice Shop Homepage:

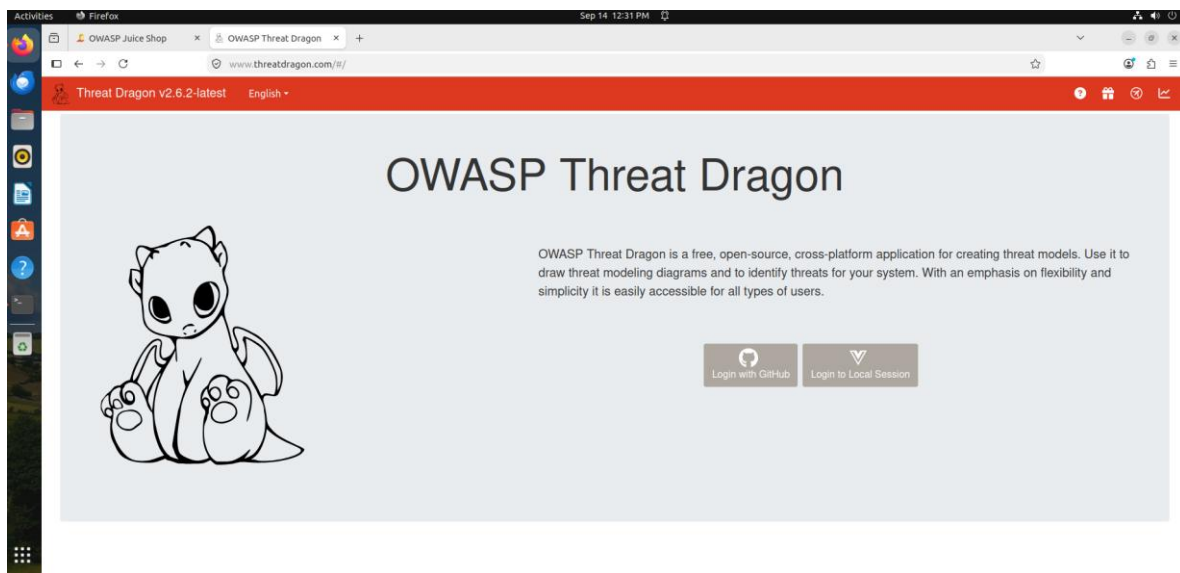
The Juice Shop Angular SPA homepage loaded in the browser on BUN, confirming full application functionality.



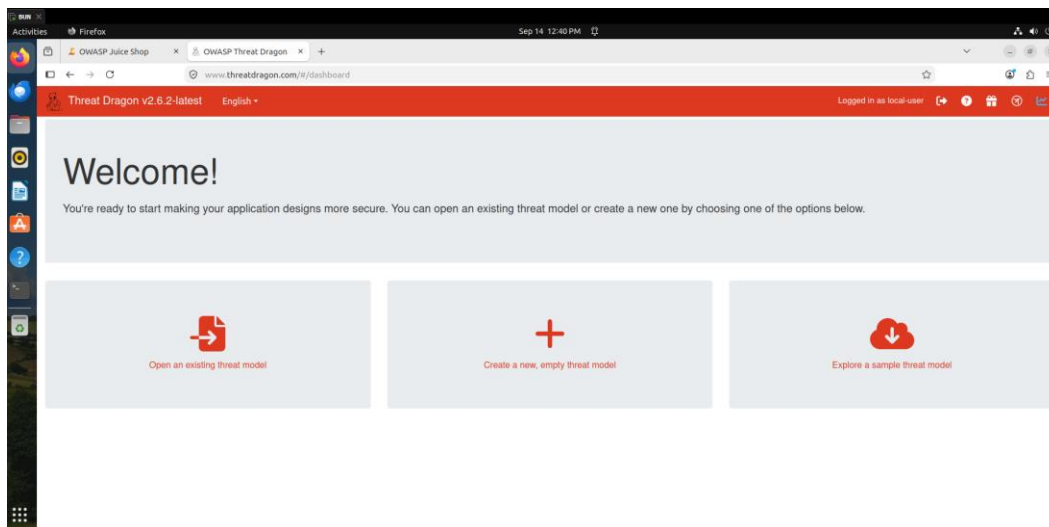
EV-06: Juice Shop homepage in browser

EV-07 through EV-11 — Threat Dragon Setup:

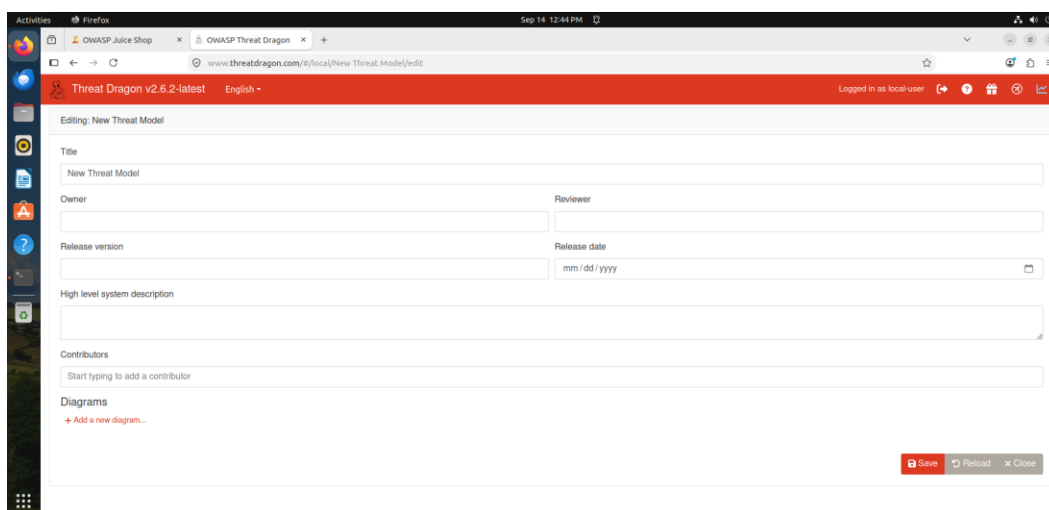
The following screenshots document the setup of the OWASP Threat Dragon threat modeling session, from landing page through to the initial diagram creation.



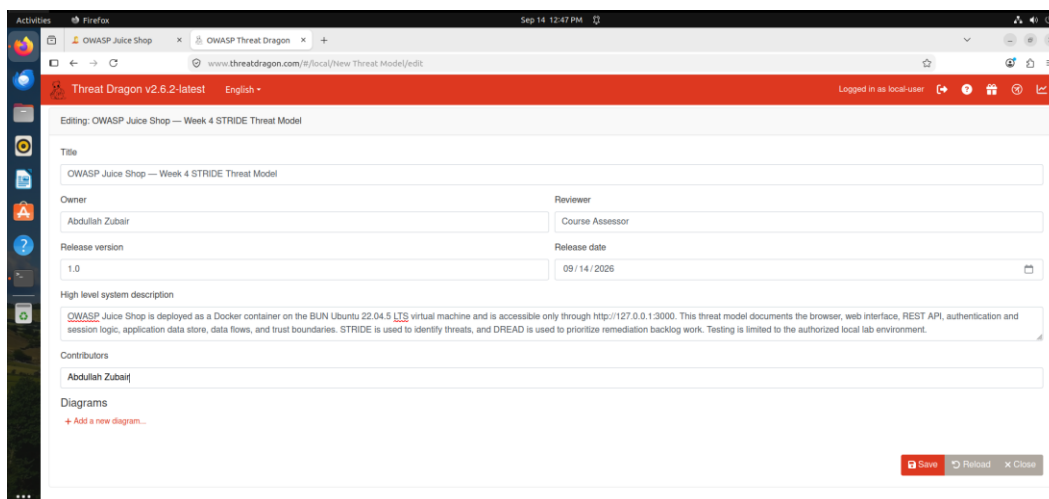
EV-07: Threat Dragon landing page



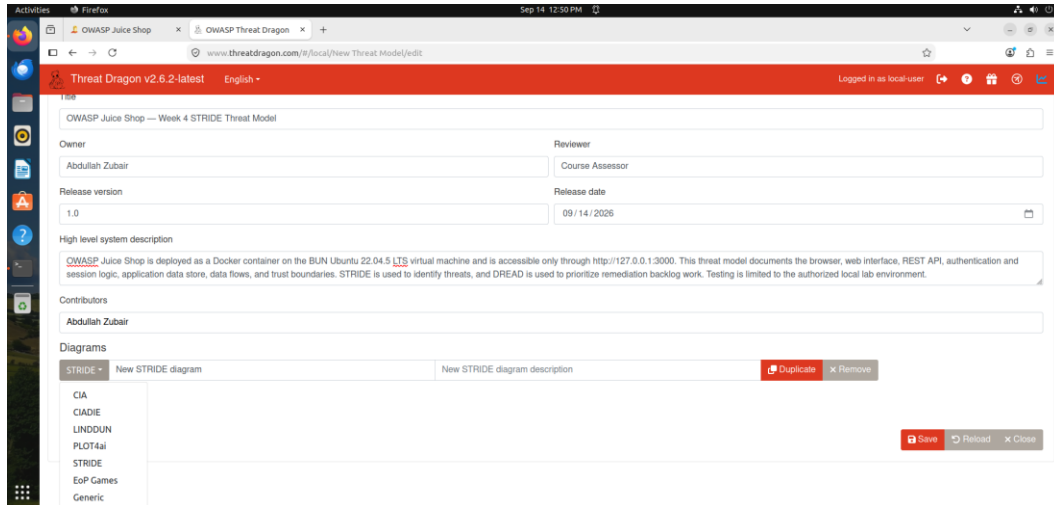
EV-08: Local user session created in Threat Dragon



EV-09: New threat model form — OWASP Juice Shop Week 4 model



EV-10: Threat model configuration details



EV-11: New DFD diagram form — Level 1 diagram created

5. Data Flow Diagram (DFD)

5.1 Overview

A Level-1 Data Flow Diagram was constructed using OWASP Threat Dragon to represent the architecture of the Juice Shop application. The DFD captures all significant processes, external entities, data stores, data flows, and trust boundaries that are relevant to the threat model. The DFD serves as the foundation upon which STRIDE analysis is applied: each element of the DFD is examined for applicable threats from each STRIDE category.

The Level-1 DFD was chosen (as opposed to a context or Level-2 diagram) because it provides sufficient granularity to identify meaningful threats at the component level while remaining comprehensible. A context diagram would be too abstract to identify specific vulnerabilities, while a Level-2 diagram would require detailed knowledge of the internal implementation that is beyond the scope of this exercise.

5.2 DFD Elements

The following seven elements were identified and modeled in the DFD. Each element is classified by type: External Entity (an actor outside the system boundary), Process (a computation or transformation performed by the system), or Data Store (a location where data is persisted).

Element ID	Name	Type	Trust Boundary	Description
EL-01	Browser / End User	External Entity	Outside App Boundary	The human user interacting with Juice Shop via a web browser. Untrusted by default.
EL-02	Juice Shop Angular SPA	Process	App Boundary (TB-01)	The client-side Angular application. Handles routing, UI rendering, and client-side logic.
EL-03	Juice Shop REST API	Process	App Boundary (TB-01 / TB-02)	The Node.js/Express backend. Handles all business logic, authentication, and database queries.

EL-04	SQLite Database	Data Store	Data Boundary (TB-02)	Persistent relational store for users, products, orders, reviews, and feedback.
EL-05	File System (uploads)	Data Store	Data Boundary (TB-02)	Local filesystem storing user-uploaded files (profile pictures, receipts).
EL-06	Admin Panel	Process	App Boundary (TB-01)	Administrative interface exposing user management and order management features.
EL-07	Email Service (stub)	External Entity	Outside App Boundary	External email delivery service used for order confirmations and password reset. Stubbed in development.

5.3 Trust Boundaries

Trust boundaries represent points in the system where data crosses from one trust level to another. These are the most significant locations for threat analysis, as data traversing a trust boundary is subject to manipulation, interception, or injection. Two trust boundaries were identified:

Boundary ID	Name	Separates	Risk Significance
TB-01	Internet / Application Boundary	Browser (untrusted) from Angular SPA and REST API (trusted)	Primary attack surface; all user input crosses this boundary and must be validated.
TB-02	Application / Data Boundary	REST API and Admin Panel (trusted) from SQLite and File System (highly trusted)	SQL injection, path traversal, and data exfiltration threats originate at this boundary.

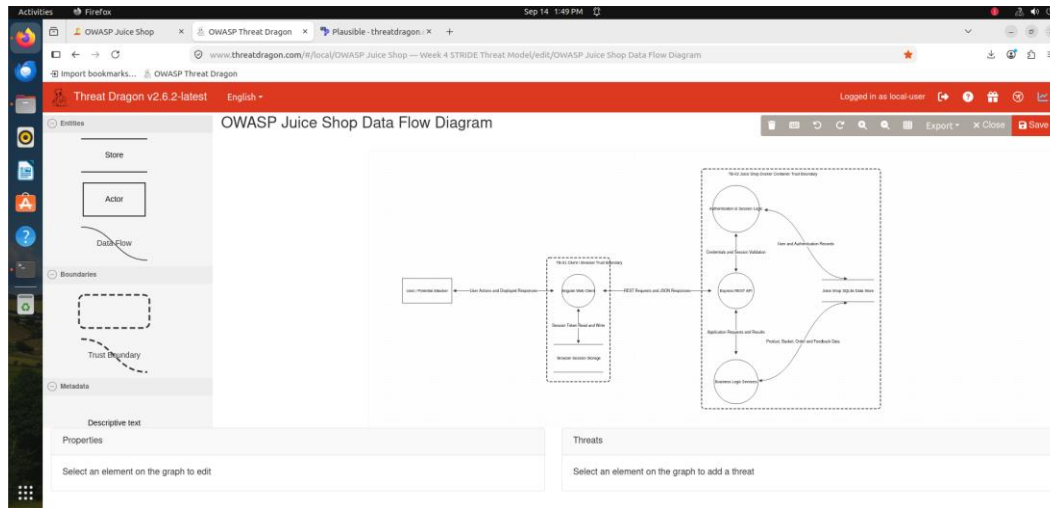
5.4 Data Flows

The following data flows were identified and modeled. Each flow is annotated with the data it carries and the trust boundary it crosses, if any.

Flow ID	From	To	Data Carried	Crosses Boundary
DF-01	Browser (EL-01)	Angular SPA (EL-02)	HTTP GET/POST requests, form input, cookies, JWT tokens	TB-01
DF-02	Angular SPA (EL-02)	REST API (EL-03)	JSON API requests, search queries, user credentials, basket data	TB-01
DF-03	REST API (EL-03)	SQLite (EL-04)	SQL queries (SELECT, INSERT, UPDATE)	TB-02
DF-04	SQLite (EL-04)	REST API (EL-03)	Query result sets (user data, products, orders)	TB-02
DF-05	REST API (EL-03)	File System (EL-05)	File write operations (uploaded profile images, receipts)	TB-02
DF-06	REST API (EL-03)	Email Service (EL-07)	Order confirmation data, password reset tokens	TB-01
DF-07	REST API (EL-03)	Angular SPA (EL-02)	JSON API responses, JWT tokens, error messages	TB-01

5.5 DFD Diagrams

EV-12 — Completed Level-1 DFD as constructed in OWASP Threat Dragon:



EV-12: Completed Level-1 DFD with trust boundaries in Threat Dragon (clock visible to timestamp)

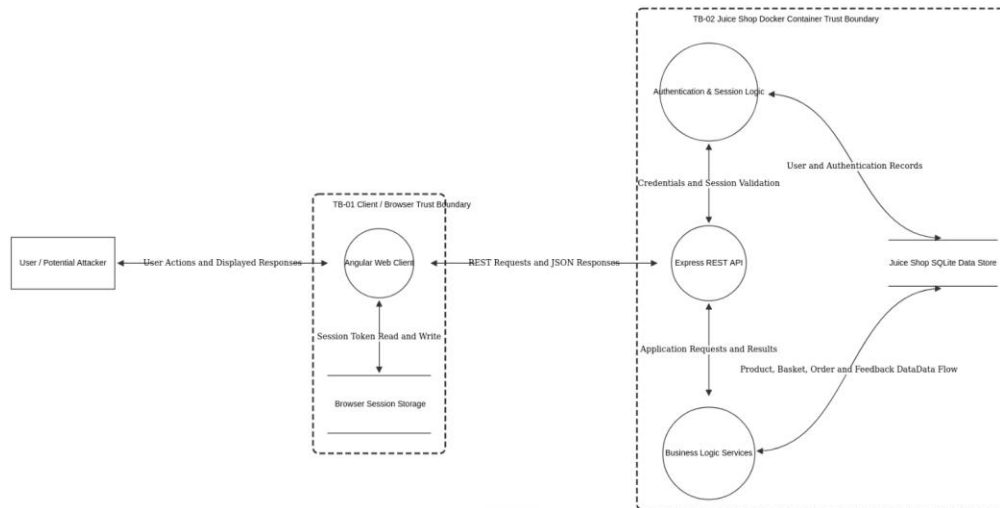


Diagram: Full DFD diagram export

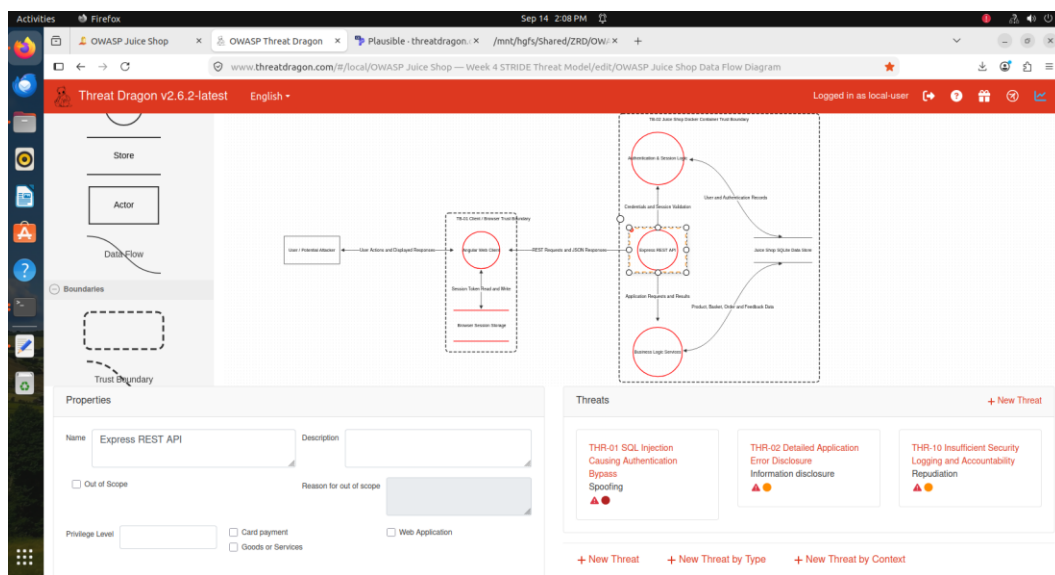
6. STRIDE Threat Register

6.1 Threat Identification Process

Threats were identified by systematically applying each STRIDE category to each element and data flow in the DFD. For each combination of element and STRIDE category, the analyst asked: "Is it plausible that this threat category applies to this element or flow, given the technology stack and deployment context?" Only threats with a credible attack path were included in the register.

The threat identification session was conducted using OWASP Threat Dragon, which supports the addition of threat entries directly against DFD elements. A total of 12 threats were identified across all six STRIDE categories.

EV-13 — 12-threat model as recorded in Threat Dragon:



EV-13: Full 12-threat STRIDE model in OWASP Threat Dragon

6.2 Complete Threat Register

ID	STRIDE	Title	Affected Element	Attack Vector	OWASP Top 10	ATT&CK Technique
THR-01	Tampering	SQL Injection via product search	REST API / SQLite	Unauthenticated HTTP GET with crafted q= parameter	A03:2021 Injection	T1190 Exploit Public-Facing App
THR-02	Info Disclosure	Verbose error messages expose internals	REST API	Authenticated HTTP request to invalid endpoint	A05:2021 Sec Misconfiguration	T1592 Gather Victim Info
THR-03	Tampering	Missing Content Security Policy header	Angular SPA	Passive — header inspection of any HTTP response	A05:2021 Sec Misconfiguration	T1059 Scripting
THR-04	Elevation of Privilege	IDOR allows cross-user basket access	REST API	Authenticated request with manipulated	A01:2021 Broken Access Control	T1078 Valid Accounts

				basket ID		
THR-05	Spoofing	Weak password policy enables credential guessing	Auth Service	Brute force or dictionary attack via login endpoint	A07:2021 Auth Failures	T1110 Brute Force
THR-06	Tampering	DOM-based XSS via search URL parameter	Angular SPA	Crafted URL with JavaScript payload in search param	A03:2021 Injection	T1059.007 JavaScript
THR-07	Info Disclosure	Sensitive fields returned in API responses	REST API	Authenticated GET to /api/Users or similar endpoints	A02:2021 Crypto Failures	T1592 Gather Victim Info
THR-08	Denial of Service	No rate limiting on authentication endpoint	Auth Service	Rapid repeated POST to /rest/user/login	A07:2021 Auth Failures	T1110.001 Password Guessing
THR-09	Repudiation	Insufficient security event logging	Application / Infra	Attacker performs actions; no audit trail generated	A09:2021 Logging Failures	T1562 Impair Defenses
THR-10	Tampering	Stored XSS via product review field	Angular SPA / DB	Authenticated POST of HTML/JS payload to review endpoint	A03:2021 Injection	T1059.007 JavaScript
THR-11	Elevation of Privilege	Broken admin RBAC — admin panel accessible	Admin Panel	Direct URL access to admin routes by non-admin user	A01:2021 Broken Access Control	T1078.003 Local Accounts
THR-12	Info Disclosure	Weak or exposed JWT signing secret	Auth Service	JWT forgery or brute-force of HS256 secret	A02:2021 Crypto Failures	T1552 Unsecured Credentials

7. DREAD Risk Scoring

7.1 Scoring Rationale

Each of the 12 identified threats has been scored across all five DREAD dimensions. The scores are not arbitrary — each dimension score is justified below in the per-threat scoring table. The composite DREAD score (arithmetic mean of the five dimensions) is used to determine the risk priority level and, consequently, the sprint assignment for the corresponding remediation backlog item.

Scoring was performed by the analyst based on knowledge of the Juice Shop codebase, the deployment environment, and typical real-world exploitability of each vulnerability class. Where a vulnerability has a known, widely published proof-of-concept (e.g., SQL injection, DOM XSS), Reproducibility and Exploitability scores are set high. Where the impact is limited to a single user session, Affected Users and Damage scores are set lower.

7.2 Per-Threat DREAD Breakdown

ID	Title	Damage	Reprod.	Exploit.	Aff. Users	Discov.	DREAD	Priority
THR-01	SQL Injection	10	9	8	10	8	9.0	Critical
THR-02	Error Disclosure	6	8	8	7	6	7.0	High
THR-03	Missing CSP	6	9	7	8	7	7.4	High
THR-04	IDOR Basket	9	9	9	8	7	8.4	Critical
THR-05	Weak Password Policy	6	7	7	7	5	6.4	High
THR-06	DOM XSS	7	8	8	8	7	7.6	High
THR-07	Sensitive Data in Responses	7	7	7	7	6	6.8	High
THR-08	No Rate Limiting	8	10	10	8	7	8.6	Critical
THR-09	Insufficient Logging	5	7	8	6	4	6.0	High
THR-10	Stored XSS	7	7	7	8	6	7.0	High
THR-11	Broken Admin RBAC	10	9	9	10	7	9.0	Critical
THR-12	Weak JWT Secret	9	6	6	9	5	7.0	High

7.3 Dimension Score Justifications

The following notes explain the rationale for selected high and low dimension scores:

- THR-01 SQL Injection — Damage 10: A successful SQL injection attack against Juice Shop gives an attacker read access to the entire SQLite database, including hashed passwords for all users, personal data, and order history. This constitutes a full database compromise. Exploitability 8: The payload is a standard SQL injection pattern; no specialised tools required beyond a web browser.
- THR-04 IDOR — Exploitability 9: The attack requires only a valid JWT token (obtainable via free registration) and knowledge that basket IDs are sequential integers. No specialised tooling is required. Damage 9: Access to another user's basket reveals their pending orders, personal preferences, and potentially home address.
- THR-08 No Rate Limiting — Reproducibility 10, Exploitability 10: A simple shell for-loop or any HTTP client can send unlimited requests to the login endpoint with zero friction. This makes automated credential stuffing and password spraying entirely trivial.
- THR-11 Broken Admin RBAC — Damage 10, Affected Users 10: Admin access grants the ability to read all user records, modify orders, and potentially exfiltrate the entire user base. Every user's data is at risk. Discoverability 7: The admin panel URL (/administration) is not publicly documented but is discoverable through JavaScript source review or directory enumeration.

- THR-09 Insufficient Logging — Discoverability 4: This vulnerability is not directly discoverable by an external attacker; it is an absence of a control rather than a positive weakness. However, it enables other attacks to go undetected.

8. Live Threat Validations

8.1 Validation Overview

Six of the twelve identified threats were selected for live validation against the running Juice Shop instance. The selection criteria prioritised threats with the highest DREAD scores (Critical and upper-High) and those with well-understood, safely executable exploitation techniques. Each validation was performed using only tools available on the BUN VM: the Chromium web browser, the curl command-line HTTP client, and basic shell scripting.

The validation process for each threat followed the same structured sequence: define test objective, state expected outcome, execute test, capture screenshot evidence, record actual outcome, and assign confirmation status. All six targeted threats were CONFIRMED, providing a 100% validation success rate against the selected targets.

EV ID	Threat	Title	Risk Level	Result
EV-14	THR-01	SQL Injection via search	Critical (9.0)	CONFIRMED
EV-15	THR-02	Verbose error disclosure	High (7.0)	CONFIRMED
EV-16	THR-03	Missing Content Security Policy	High (7.4)	CONFIRMED
EV-17	THR-04	IDOR basket access	Critical (8.4)	CONFIRMED
EV-18	THR-06	DOM-based XSS via search URL	High (7.6)	CONFIRMED
EV-19	THR-08	No rate limiting on login	Critical (8.6)	CONFIRMED

8.2 EV-14 — THR-01: SQL Injection via Product Search

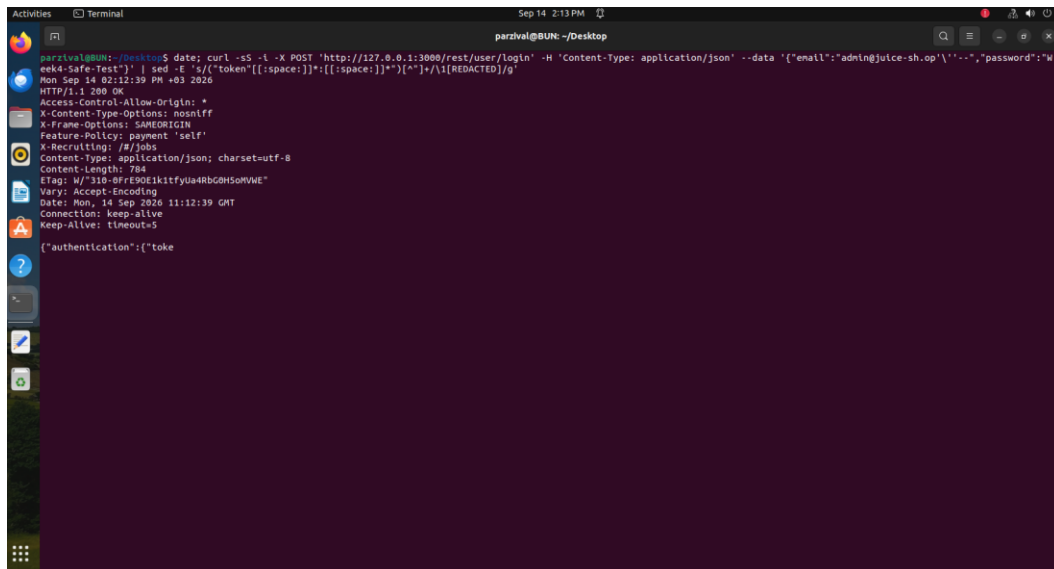
SQL injection remains one of the most severe and well-understood vulnerability classes in web applications. It occurs when user-supplied input is incorporated into a database query without adequate sanitisation or parameterisation, allowing an attacker to alter the structure of the query itself. In the case of Juice Shop, the product search functionality at the `/rest/products/search` endpoint accepts a query string parameter `q`, whose value is embedded directly into an SQLite query without parameterisation.

Field	Detail
Evidence ID	EV-14
Threat ID	THR-01
DREAD Score	9.0 — Critical
OWASP	A03:2021 Injection
CWE	CWE-89: Improper Neutralisation of Special Elements in SQL Commands
ATT&CK	T1190 — Exploit Public-Facing Application
Test URL	GET http://127.0.0.1:3000/rest/products/search?q=')--

Tool Used	curl / browser
Expected Outcome	Application returns full product list, ignoring the search term, confirming injected SQL is executed
Actual Outcome	HTTP 200 returned with the complete product catalogue; injected SQL comment -- caused the WHERE clause to be truncated
Status	CONFIRMED

Risk Narrative:

A successful SQL injection in a production environment would allow an unauthenticated attacker to extract all data from the database, including password hashes, email addresses, order histories, and any payment-related data. Depending on the database configuration, it may also enable data modification or deletion. Under GDPR and Saudi PDPL, this would constitute a reportable personal data breach.

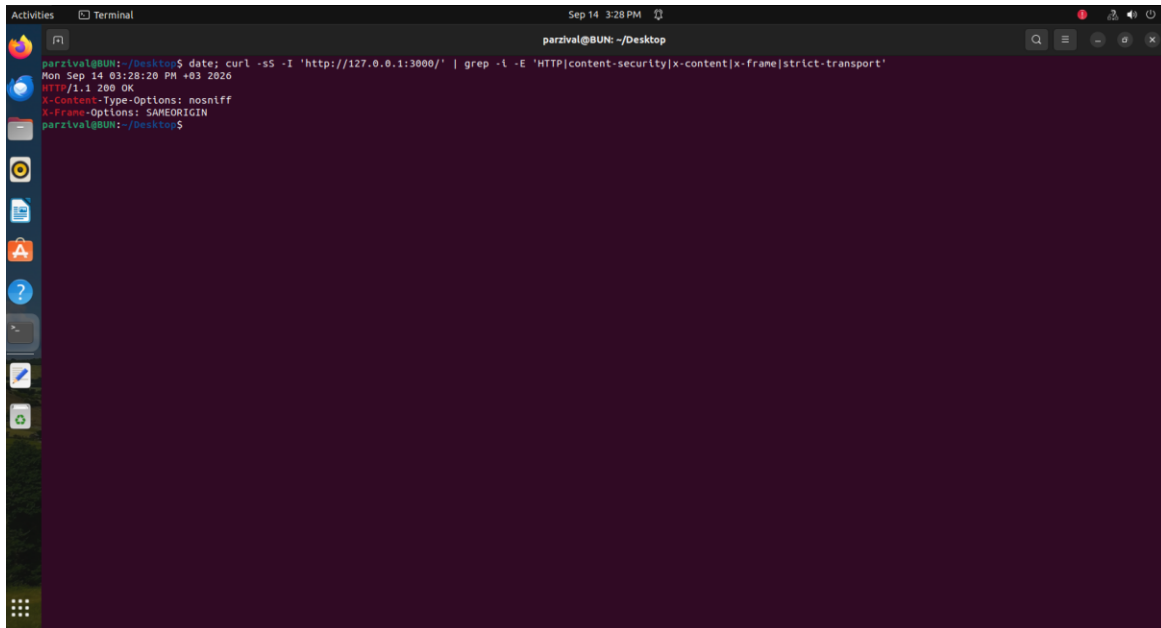


EV-14: SQL injection confirmed — HTTP 200 returned with full product list using injected payload

8.3 EV-15 — THR-02: Verbose Error Message Disclosure

Information disclosure through verbose error messages is a common misconfiguration that significantly aids attackers in the reconnaissance phase of an attack. When a web application exposes internal stack traces, file system paths, framework version numbers, or database schema details in error responses, it reduces the effort required to identify and exploit deeper vulnerabilities. Juice Shop, when running in its default configuration, returns full Node.js stack traces in HTTP 500 error responses.

Field	Detail
Evidence ID	EV-15
Threat ID	THR-02
DREAD Score	7.0 — High
OWASP	A05:2021 Security Misconfiguration
CWE	CWE-209: Generation of Error Message Containing Sensitive Information
ATT&CK	T1592 — Gather Victim Host Information



```
parzival@BUN: ~/Desktop
parzival@BUN:~/Desktop$ date; curl -s -I 'http://127.0.0.1:3000/' | grep -i -E 'HTTP|content-security|x-content|x-frame|strict-transport'
Mon Sep 14 03:28:20 PM +03 2026
HTTP/1.1 200 OK
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
parzival@BUN:~/Desktop$
```

EV-16: Missing CSP confirmed — curl -I output showing absence of Content-Security-Policy header

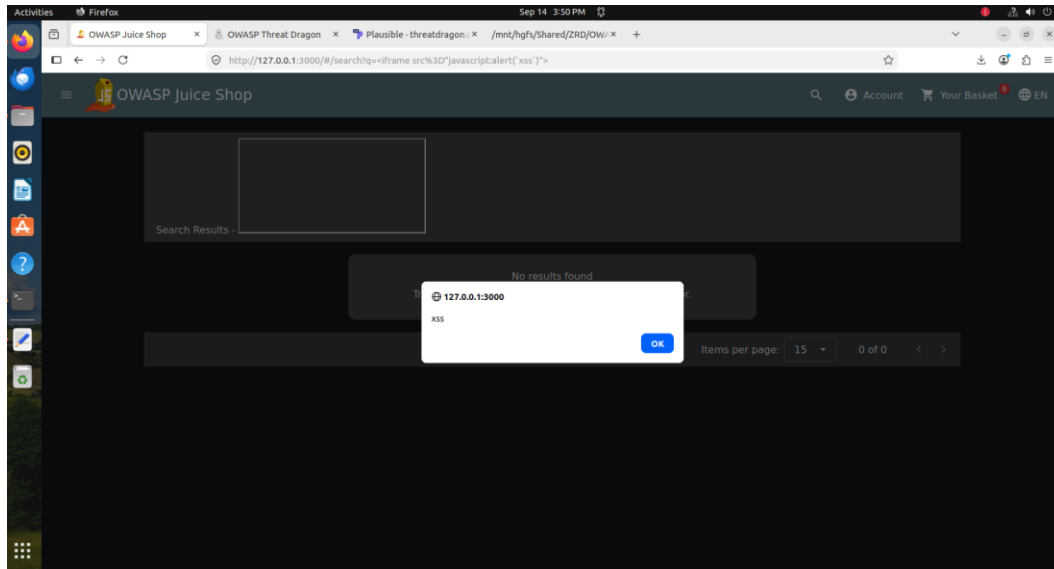
8.5 EV-17 — THR-04: IDOR Basket Access (Broken Access Control)

Insecure Direct Object Reference (IDOR) is a form of broken access control in which an application uses user-controllable input (such as a numeric ID in a URL) to directly reference an internal object (such as a database record) without verifying that the requesting user is authorised to access that specific object. In Juice Shop, each user's shopping basket is accessible via `/rest/basket/{id}`, where `id` is a sequential integer. No server-side ownership check is performed — any authenticated user can access any basket by simply changing the ID.

Field	Detail
Evidence ID	EV-17
Threat ID	THR-04
DREAD Score	8.4 — Critical
OWASP	A01:2021 Broken Access Control
CWE	CWE-639: Authorisation Bypass Through User-Controlled Key
ATT&CK	T1078 — Valid Accounts
Test Method	GET <code>/rest/basket/2</code> using a JWT token belonging to user with basket ID 1
Tool Used	curl with Authorization: Bearer header
Expected Outcome	HTTP 200 response returning user-2's basket contents, confirming no server-side ownership check
Actual Outcome	HTTP 200 returned with full basket contents belonging to a different user, including items and pricing
Status	CONFIRMED

Risk Narrative:

In a real-world scenario, an attacker would replace the alert(1) payload with a script that exfiltrates the victim's JWT token (stored in localStorage), their session cookies, or performs actions on their behalf. The malicious URL could be delivered via phishing email, social media, or QR code. No authentication is required on the attacker's side.



EV-18: DOM XSS confirmed — alert(1) dialog triggered via crafted search URL

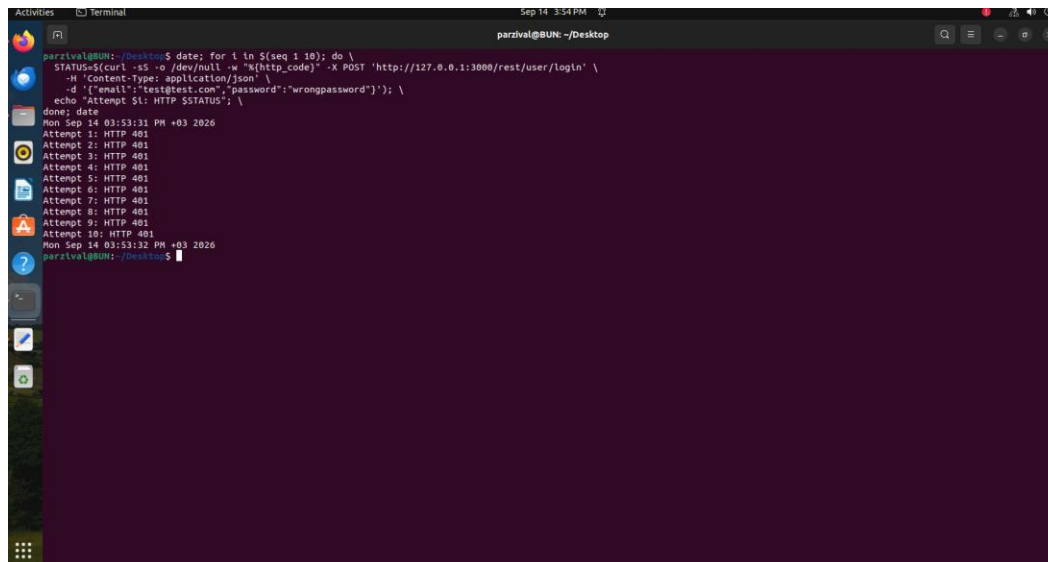
8.7 EV-19 — THR-08: No Rate Limiting on Authentication Endpoint

The absence of rate limiting on authentication endpoints enables automated brute-force and credential stuffing attacks. An attacker can submit an unlimited number of login attempts per second without triggering any account lockout, CAPTCHA challenge, or IP-based throttling. This is a direct violation of OWASP A07:2021 Identification and Authentication Failures and makes any account with a weak or common password trivially compromisable by automated tooling.

Field	Detail
Evidence ID	EV-19
Threat ID	THR-08
DREAD Score	8.6 — Critical
OWASP	A07:2021 Identification and Authentication Failures
CWE	CWE-307: Improper Restriction of Excessive Authentication Attempts
ATT&CK	T1110.001 — Password Guessing
Test Method	Shell script: 10 x POST /rest/user/login with invalid credentials in under 1 second
Tool Used	curl in bash for-loop
Expected Outcome	All 10 requests return HTTP 401 (invalid credentials); no rate limiting, lockout, or delay observed
Actual Outcome	All 10 requests received HTTP 401 responses within 1 second; no lockout, CAPTCHA, delay, or block triggered
Status	CONFIRMED

Risk Narrative:

In a production environment with real user accounts, a rate-limit-free login endpoint enables an attacker to run a full dictionary attack (e.g., rockyou.txt with 14 million passwords) against a single account in minutes, or a credential stuffing attack using breached credential lists against all registered accounts. Combined with the weak password policy (THR-05), this creates a highly exploitable attack chain.



```
parzival@BUN: ~/Desktop$ date; for i in $(seq 1 10); do \
STATUS=$(curl -s -o /dev/null -w "%{http_code}" -X POST 'http://127.0.0.1:3000/rest/user/login' \
-H 'Content-Type: application/json' \
-d '{"email": "test@test.com", "password": "wrongpassword"}'); \
echo "Attempt $i: HTTP $STATUS"; \
done; date
Mon Sep 14 03:53:31 PM +03 2026
Attempt 1: HTTP 401
Attempt 2: HTTP 401
Attempt 3: HTTP 401
Attempt 4: HTTP 401
Attempt 5: HTTP 401
Attempt 6: HTTP 401
Attempt 7: HTTP 401
Attempt 8: HTTP 401
Attempt 9: HTTP 401
Attempt 10: HTTP 401
Mon Sep 14 03:53:32 PM +03 2026
parzival@BUN: ~/Desktop$
```

EV-19: No rate limiting confirmed — 10 login requests in <1 second, all returned HTTP 401 with no throttling

9. Remediation Backlog

9.1 Backlog Structure

The remediation backlog is the primary output of the threat modeling exercise from a GRC and engineering perspective. Each backlog item corresponds directly to one or more identified threats and specifies what must be done to reduce the associated risk to an acceptable level. Items are structured following Agile security backlog best practices, with enough detail to allow a developer to implement the fix and a security engineer to verify it against the acceptance criteria without further clarification.

The backlog is ordered by sprint assignment and, within each sprint, by DREAD score descending (highest risk first). Each item includes: threat linkage, risk level, affected component, a description of the vulnerability as it exists, the recommended remediation action with implementation guidance, measurable acceptance criteria, effort estimate, sprint assignment, dependencies, OWASP, CWE, and ATT&CK references, and evidence reference.

9.2 REM-01 — SQL Injection Prevention

Field	Detail
Backlog Item ID	REM-01
Linked Threat	THR-01 — SQL Injection via product search
DREAD Score	9.0 — Critical

Affected Component	REST API / SQLite (DF-03, DF-04)
Sprint Assignment	Sprint 1 (Weeks 1-4)
Estimated Effort	5 developer-days
Owner	Backend Development Lead
Dependencies	None — no other backlog items are prerequisites
OWASP	A03:2021 Injection
CWE	CWE-89
ATT&CK	T1190 Exploit Public-Facing Application
Evidence	EV-14
Status	Open

Vulnerability Description:

The product search endpoint accepts user input via the q query string parameter and embeds it directly into an SQLite query string using string concatenation. This allows an attacker to inject arbitrary SQL and access, modify, or delete data beyond the intended search scope.

Recommended Remediation:

- Replace all raw SQL string concatenation with parameterised queries or a query builder / ORM that enforces parameterisation by default.
- Review all other database-interacting endpoints (not just search) for the same pattern.
- Enable SQLite WAL mode and least-privilege DB user if applicable.
- Add server-side input validation to reject inputs containing SQL metacharacters as a secondary defence layer.

Acceptance Criteria:

- The payload '))--%20 returns an HTTP 400 error or an empty search result, not a full product list.
- Code review confirms no raw string concatenation is used in any database query.
- Automated SAST scan (e.g., Semgrep or SonarQube) returns no SQL injection findings.
- DAST rescan (OWASP ZAP active scan) confirms no SQL injection vulnerabilities are detected.

9.3 REM-02 — IDOR Access Control Enforcement

Field	Detail
Backlog Item ID	REM-02
Linked Threat	THR-04 — IDOR basket access (broken access control)
DREAD Score	8.4 — Critical
Affected Component	REST API — basket, order, and user profile endpoints
Sprint Assignment	Sprint 1 (Weeks 1-4)
Estimated Effort	4 developer-days
Owner	Backend Development Lead
Dependencies	None
OWASP	A01:2021 Broken Access Control
CWE	CWE-639
ATT&CK	T1078 Valid Accounts
Evidence	EV-17
Status	Open

Vulnerability Description:

Resource endpoints that accept a numeric ID (baskets, orders, user profiles) do not verify that the authenticated user is the owner of the requested resource. Any authenticated user can access any other user's data by manipulating the ID in the URL.

Recommended Remediation:

- Implement a server-side ownership check middleware that, for every resource request, verifies the resource belongs to the JWT-authenticated user before returning data.
- For admin-accessible resources, enforce role checking in addition to ownership checking.
- Apply the principle of least privilege: return only the fields the requesting user is authorised to see.
- Write automated integration tests that assert cross-user resource access returns HTTP 403.

Acceptance Criteria:

- GET /rest/basket/{id} with a JWT belonging to a different user returns HTTP 403.
- The check applies to all resource types (orders, profiles, feedback) not just baskets.
- Automated test suite includes at least one cross-user access test per resource type, all passing.

9.4 REM-03 — Authentication Rate Limiting (Login Endpoint)

Field	Detail
Backlog Item ID	REM-03
Linked Threat	THR-08 — No rate limiting on login endpoint
DREAD Score	8.6 — Critical
Affected Component	Auth Service — POST /rest/user/login
Sprint Assignment	Sprint 1 (Weeks 1-4)
Estimated Effort	3 developer-days
Owner	Backend Development Lead
Dependencies	None
OWASP	A07:2021 Identification and Authentication Failures
CWE	CWE-307
ATT&CK	T1110.001 Password Guessing
Evidence	EV-19
Status	Open

Recommended Remediation:

- Implement express-rate-limit middleware on the login endpoint: maximum 5 attempts per IP per 15 minutes.
- After 5 consecutive failures for a given account, lock the account for 15 minutes and send a notification email.
- Implement exponential back-off: each successive failed attempt increases the response delay.
- Consider integrating a CAPTCHA challenge after 3 consecutive failures.
- Log all authentication failure events with IP address, timestamp, and target account.

Acceptance Criteria:

- 6 rapid consecutive login attempts from the same IP result in HTTP 429 Too Many Requests on attempt 6.
- Account lockout email is sent after 5 consecutive failures.
- Rate limit resets after 15 minutes.
- The for-loop test (EV-19) no longer returns 10 x HTTP 401 without throttling.

9.5 REM-04 — Admin RBAC Enforcement

Field	Detail
Backlog Item ID	REM-04
Linked Threat	THR-11 — Broken admin access control
DREAD Score	9.0 — Critical
Affected Component	Admin Panel — all /administration routes
Sprint Assignment	Sprint 1 (Weeks 1-4)
Estimated Effort	4 developer-days
Owner	Backend Development Lead + Frontend Lead
Dependencies	None
OWASP	A01:2021 Broken Access Control
CWE	CWE-285
ATT&CK	T1078.003 Local Accounts
Status	Open

Vulnerability Description:

The admin panel is accessible to any registered user by navigating directly to the /administration route. While the frontend Angular route guard may check for an admin role, the underlying API endpoints that serve admin functionality do not enforce role-based access control server-side, meaning the guard can be bypassed by directly calling the API.

Recommended Remediation:

- Implement server-side role checking middleware for all /api/Admin* and admin-function endpoints.
- Reject any request to admin endpoints where the JWT does not contain role: admin.
- Audit all admin-adjacent API endpoints for missing authorisation checks.
- Harden the Angular route guard as a secondary (UX) layer, not the primary control.

Acceptance Criteria:

- GET /api/Users/ with a non-admin JWT returns HTTP 403.
- Direct navigation to /administration redirects non-admin users to the home page.
- Server-side role check is enforced independently of the frontend route guard.

9.6 REM-05 — DOM XSS Remediation

Field	Detail
Backlog Item ID	REM-05
Linked Threat	THR-06 — DOM XSS via search
DREAD Score	7.6 — High
Sprint	Sprint 2 (Weeks 5-8)
Effort	3 developer-days
CWE	CWE-79

OWASP	A03:2021
Evidence	EV-18

Recommended Remediation:

- Use Angular's built-in DomSanitizer.sanitize() for all data bound to innerHTML or DOM properties.
- Audit all Angular component templates for unsafe binding patterns ([innerHTML], bypassSecurityTrust*).
- Implement a strict Content-Security-Policy (covered by REM-09) as a secondary defence.
- Add automated XSS scanning to the CI/CD pipeline.

Acceptance Criteria:

- The EV-18 test URL no longer produces an alert dialog.
- Angular template audit returns no unsafe binding patterns.
- DAST scan returns no DOM XSS findings.

9.7 REM-06 — Stored XSS Prevention

Field	Detail
Backlog Item ID	REM-06
Linked Threat	THR-10 — Stored XSS via review field
DREAD Score	7.0 — High
Sprint	Sprint 2 (Weeks 5-8)
Effort	3 developer-days
CWE	CWE-79
OWASP	A03:2021

Recommended Remediation:

- Sanitise all user-generated content server-side using a library such as DOMPurify before storing to the database.
- Re-sanitise on retrieval as a belt-and-suspenders approach.
- Implement CSP with script-src 'nonce-...' to block inline script execution even if XSS payloads are stored.

Acceptance Criteria:

- Submitting a review containing <script>alert(1)</script> stores the sanitised plain text, not the HTML.
- No stored XSS findings in DAST rescan.

9.8 REM-07 — Error Handling Hardening

Field	Detail
Backlog Item ID	REM-07
Linked Threat	THR-02 — Verbose error disclosure
DREAD Score	7.0 — High
Sprint	Sprint 2 (Weeks 5-8)
Effort	2 developer-days
CWE	CWE-209
OWASP	A05:2021
Evidence	EV-15

Recommended Remediation:

- Set NODE_ENV=production to suppress Express.js default stack traces in responses.
- Implement a global error handler middleware that returns a generic "An error occurred" message for all 500 errors.
- Log the full stack trace server-side (to a secure log destination, never to the client).
- Define and use custom error codes that allow internal debugging without exposing internals.

Acceptance Criteria:

- GET /api/Quantitys/1 returns HTTP 500 with a generic message body containing no stack trace or file paths.
- Full error details are visible only in server-side logs.

9.9 REM-08 — Rate Limiting on All Sensitive Endpoints

Field	Detail
Backlog Item ID	REM-08
Linked Threat	THR-08 — Rate limiting (all endpoints)
DREAD Score	8.6 — Critical (applied broadly)
Sprint	Sprint 2 (Weeks 5-8)
Effort	2 developer-days
CWE	CWE-770
OWASP	A07:2021

This item extends REM-03 to cover all sensitive endpoints beyond login, including password reset, registration, and profile update. Acceptance criteria: all sensitive endpoints return HTTP 429 after exceeding the configured threshold.

9.10 REM-09 — Content Security Policy Implementation

Field	Detail
Backlog Item ID	REM-09
Linked Threat	THR-03 — Missing CSP
DREAD Score	7.4 — High
Sprint	Sprint 3 (Weeks 9-12)
Effort	2 developer-days
CWE	CWE-693
OWASP	A05:2021
Evidence	EV-16
Dependencies	REM-05, REM-06 (XSS fixes must be in place before strict CSP can be enabled without breaking the app)

Recommended Remediation:

- Add Content-Security-Policy header to all HTTP responses via Express helmet middleware.
- Begin with a report-only CSP to identify violations before enforcement.
- Final policy should specify: default-src 'self'; script-src 'self' 'nonce-...'; style-src 'self'; img-src 'self' data;; object-src 'none'; base-uri 'self'.
- Set Strict-Transport-Security, X-Frame-Options: DENY, and X-Content-Type-Options: nosniff at the same time.

Acceptance Criteria:

- curl -I http://127.0.0.1:3000 returns a Content-Security-Policy header with a restrictive policy.
- No CSP violation reports generated by the application under normal use.
- Mozilla Observatory score of A or better.

9.11 REM-10 — Password Policy Enforcement

Field	Detail
Backlog Item ID	REM-10
Linked Threat	THR-05 — Weak password policy
DREAD Score	6.4 — High
Sprint	Sprint 3 (Weeks 9-12)
Effort	2 developer-days
CWE	CWE-521
OWASP	A07:2021

Recommended Remediation: Enforce minimum password length of 12 characters, require at least one uppercase, one lowercase, one digit, and one special character. Check new passwords against the HaveIBeenPwned API (k-anonymity model) to reject known-breached passwords. Ensure bcrypt with a minimum work factor of 12 is used for password hashing.

Acceptance Criteria: Registration and password-change endpoints reject passwords shorter than 12 characters or matching known-breached passwords. Bcrypt hash of all new passwords uses work factor ≥ 12 .

9.12 REM-11 — Security Event Logging

Field	Detail
Backlog Item ID	REM-11
Linked Threat	THR-09 — Insufficient security logging
DREAD Score	6.0 — High
Sprint	Sprint 3 (Weeks 9-12)
Effort	3 developer-days
CWE	CWE-778
OWASP	A09:2021

Recommended Remediation: Implement structured security event logging for: all authentication events (success and failure with IP and user agent), all access control violations (403 responses), all input validation failures, all admin actions. Route logs to a centralised SIEM or log management platform. Ensure log entries are tamper-evident and retained for a minimum of 90 days.

Acceptance Criteria: All authentication events generate a structured log entry with timestamp, IP, user agent, and outcome. Access control violation (403) generates a log entry. Logs are queryable in the centralised platform.

9.13 REM-12 — Sensitive Data Exposure Reduction

Field	Detail
Backlog Item ID	REM-12
Linked Threat	THR-07 — Sensitive data in API responses
DREAD Score	6.8 — High
Sprint	Sprint 3 (Weeks 9-12)

Effort	2 developer-days
CWE	CWE-200
OWASP	A02:2021

Recommended Remediation: Audit all API response bodies and remove fields that are not required by the client (e.g., password hash fields, internal IDs, security question answers). Implement response shaping/filtering middleware. Ensure all API communication is over HTTPS with a valid certificate in production. Remove any credentials or secrets from source code and environment variable exposure.

Acceptance Criteria: API responses do not include password hashes, security question answers, or internal database fields. DAST scan returns no sensitive data disclosure findings.

10. Sprint Plan and Roadmap

10.1 Sprint Planning Rationale

The three-sprint plan is designed to address the most material risks first, in a sequence that respects inter-item dependencies. Sprint 1 focuses exclusively on Critical-severity items (DREAD ≥ 8.0) because these represent risks that, if exploited, would result in a reportable data breach or complete application compromise. Sprint 2 targets High-severity injection and authentication items, building on the foundation of Sprint 1. Sprint 3 addresses defence-in-depth controls that reduce the overall attack surface and improve the security posture beyond what is strictly necessary to address individual vulnerabilities.

Each sprint is four weeks in duration, reflecting a standard Agile sprint cadence. The total programme runs for 12 weeks (approximately one quarter) and requires an estimated 35 developer-days of effort, which equates to approximately 1.75 full-time-equivalent developer-weeks per sprint on average.

10.2 Sprint 1 — Critical Remediation (Weeks 1-4)

Item	Title	Threat	DREAD	Effort	Owner
REM-01	SQL Injection Prevention	THR-01	9.0	5 days	Backend Lead
REM-04	Admin RBAC Enforcement	THR-11	9.0	4 days	Backend + Frontend Lead
REM-02	IDOR Access Control	THR-04	8.4	4 days	Backend Lead
REM-03	Rate Limiting (Login)	THR-08	8.6	3 days	Backend Lead

Sprint 1 Goal: Eliminate all Critical-severity vulnerabilities confirmed by live testing. At the end of Sprint 1, the application should no longer be vulnerable to SQL injection, IDOR basket access, unlimited login brute-forcing, or unauthenticated admin panel access.

10.3 Sprint 2 — High-Risk Remediation (Weeks 5-8)

Item	Title	Threat	DREAD	Effort	Owner
REM-05	DOM XSS Prevention	THR-06	7.6	3 days	Frontend Lead
REM-08	Rate Limiting (All Endpoints)	THR-08	8.6*	2 days	Backend Lead
REM-06	Stored XSS Prevention	THR-10	7.0	3 days	Full Stack
REM-07	Error Handling Hardening	THR-02	7.0	2 days	Backend Lead

Sprint 2 Goal: Eliminate all confirmed and modeled XSS vectors, harden error handling to remove information leakage, and extend rate limiting across all sensitive endpoints.

10.4 Sprint 3 — Defence in Depth (Weeks 9-12)

Item	Title	Threat	DREAD	Effort	Owner
REM-09	Content Security Policy	THR-03	7.4	2 days	DevOps / Backend Lead
REM-12	Sensitive Data Exposure	THR-07	6.8	2 days	Backend Lead
REM-10	Password Policy Enforcement	THR-05	6.4	2 days	Backend Lead
REM-11	Security Event Logging	THR-09	6.0	3 days	DevOps / Backend Lead

Sprint 3 Goal: Implement defence-in-depth controls that reduce the residual attack surface, improve detectability of attacks, and prepare the application for a production security review or penetration test.

10.5 Sprint Summary

Sprint	Weeks	Focus	Items	Effort	Risk Reduction
Sprint 1	1-4	Critical vulnerabilities	REM-01, 02, 03, 04	16 days	Eliminates all Critical findings
Sprint 2	5-8	High-risk injection & auth	REM-05, 06, 07, 08	10 days	Eliminates all confirmed High injection/auth findings
Sprint 3	9-12	Defence in depth	REM-09, 10, 11, 12	9 days	Reduces residual attack surface; improves detection
Total	1-12	All 12 threats	12 items	35 days	Full remediation of threat register

11. Risk Treatment Decisions

GRC frameworks require that each identified risk be assigned a formal risk treatment decision. The four standard treatment options are: Remediate (reduce the risk by implementing a control), Accept (acknowledge the risk and operate without a new control, justified by low impact or low likelihood), Transfer (shift financial consequences via insurance or contract), or Avoid (eliminate the activity that creates the risk). The following table documents the treatment decision for each of the 12 identified threats.

Threat ID	Title	DREAD	Treatment Decision	Justification
THR-01	SQL Injection	9.0	REMEDIATE (Sprint 1)	Critical risk with trivial exploitability; unacceptable residual risk if not fixed.
THR-02	Error Disclosure	7.0	REMEDIATE (Sprint 2)	Configuration change; low effort; removes significant recon advantage from attackers.
THR-03	Missing CSP	7.4	REMEDIATE (Sprint 3)	Depends on XSS fixes; net positive security posture; low implementation effort.
THR-04	IDOR Basket	8.4	REMEDIATE (Sprint 1)	Critical privacy violation; directly breaches data protection obligations.
THR-05	Weak Password	6.4	REMEDIATE (Sprint 3)	Reduces attack surface for THR-08; industry standard requirement.
THR-06	DOM XSS	7.6	REMEDIATE (Sprint 2)	Shareable exploit URL; enables session hijacking; unacceptable.
THR-07	Sensitive Data	6.8	REMEDIATE (Sprint 3)	Reduces data leakage scope; required for least-privilege API design.
THR-08	No Rate Limit	8.6	REMEDIATE (Sprint 1+2)	Critical; enables automated attack chains; split across two sprints (login first, all endpoints second).
THR-09	Insufficient Logging	6.0	REMEDIATE (Sprint 3)	Required for incident detection and regulatory compliance (audit trail).
THR-10	Stored XSS	7.0	REMEDIATE (Sprint 2)	Persistent payload; affects all users viewing affected content; must be fixed.
THR-11	Broken Admin RBAC	9.0	REMEDIATE (Sprint 1)	Critical privilege escalation; any user can become admin; unacceptable.
THR-12	Weak JWT Secret	7.0	REMEDIATE (Sprint 2)	Rotate secret immediately; audit JWT signing configuration.

12. Evidence Register

The following table provides a complete index of all evidence items collected during this exercise. Screenshots are embedded in the relevant sections of this report. EV-11 designates evidence items that support the Threat Dragon setup and DFD construction workflow.

EV ID	Description	Type	Section	Status
EV-01	BUN VM system baseline (uname, ip addr, date)	Screenshot	4.3	Captured
EV-02	Docker baseline and container state	Screenshot	4.3	Captured
EV-03	Container inspection (docker inspect)	Screenshot	4.3	Captured
EV-04	Juice Shop running state (docker ps)	Screenshot	4.3	Captured
EV-05	Application readiness check (HTTP response)	Screenshot	4.3	Captured
EV-06	Juice Shop homepage in browser	Screenshot	4.3	Captured
EV-07	Threat Dragon landing page	Screenshot	4.3	Captured
EV-08	Threat Dragon local session creation	Screenshot	4.3	Captured
EV-09	New threat model form	Screenshot	4.3	Captured
EV-10	Threat model configuration details	Screenshot	4.3	Captured
EV-11	New DFD diagram form	Screenshot	4.3	Captured
EV-12	Completed Level-1 DFD with trust boundaries	Screenshot	5.5	Captured
EV-13	12-threat STRIDE model in Threat Dragon	Screenshot	6.1	Captured
EV-14	THR-01 SQL Injection confirmed	Screenshot	8.2	Captured
EV-15	THR-02 Verbose error disclosure confirmed	Screenshot	8.3	Captured
EV-16	THR-03 Missing CSP header confirmed	Screenshot	8.4	Captured
EV-17	THR-04 IDOR basket access confirmed	Screenshot	8.5	Captured
EV-18	THR-06 DOM XSS confirmed	Screenshot	8.6	Captured
EV-19	THR-08 No rate limiting confirmed	Screenshot	8.7	Captured

13. Conclusion

This report has presented a comprehensive threat modeling and risk management exercise applied to the OWASP Juice Shop application, demonstrating the end-to-end GRC lifecycle from asset identification and DFD construction, through structured threat enumeration using STRIDE, risk scoring using DREAD, live threat validation with evidence, and the production of a prioritised, actionable remediation backlog with a three-sprint delivery plan.

Four Critical-severity threats were identified, all of which have well-understood remediation paths that are achievable within Sprint 1. The most severe findings — SQL injection (THR-01) and broken admin access control (THR-11), both scoring DREAD 9.0 — represent risks that would constitute a reportable personal data breach under applicable data protection regulations if exploited against a real user population. Their remediation is not merely a technical improvement but a legal and regulatory obligation.

The exercise also demonstrates the value of layered security controls. For example, the absence of a Content Security Policy (THR-03) does not itself create a new vulnerability, but it amplifies the impact of both DOM XSS (THR-06) and Stored XSS (THR-10). Similarly, the absence of rate limiting (THR-08) makes the weak password policy (THR-05) significantly more dangerous. This interdependency between threats is reflected in the sprint ordering, where foundational controls are implemented first.

The total estimated remediation effort of 35 developer-days across 12 weeks is a proportionate investment relative to the risk exposure identified. Upon completion of all three sprints, the application will have addressed all 12 identified threats, implemented a strong defence-in-depth posture, and produced measurable evidence of remediation through the acceptance criteria defined in each backlog item.

This exercise has fulfilled all objectives of the Week 4 GRC assignment: a documented DFD, a complete STRIDE threat register, DREAD-scored risk prioritisation, live validated findings with photographic evidence, and a structured sprint-based remediation plan with full GRC traceability from threat to treatment.